

Longrevalidatiedashboard

Realisatiedocument

Maarten Lommers

Student Bachelor in de Toegepaste Informatica – Applicatieontwikkeling

Inhoudsopgave

1. INLEIDING	4
2. ANALYSE	5
2.1. Context	5
2.2. Scope	5
2.3. Frontend versus backend: het huismodel	5
2.5. Keuze van technologieën	6
2.5.1. Frontend: React	6
2.5.2. Backend: .NET	6
3. FRONTEND ARCHITECTUUR	7
3.1. Architecturale lagen	7
3.2. Projectstructuur (vereenvoudigd)	9
3.3. Authenticatie & Autorisatie	9
3.4. Notabel component	11
3.4.1. ProtectedRoute	11
3.5. Codeerconventies	11
4. BACKEND ARCHITECTUUR	12
4.1. Architecturale lagen	12
4.1.1. Middleware-laag	12
4.1.2. API-laag (Controller)	12
4.1.3. Applicatielaag (CQRS, Handlers, Services)	13
4.1.4. Domeinlaag	14
4.1.5. Infrastructuurlaag	14
4.1.6. Projectstructuur (vereenvoudigd)	16
4.1.7. Toegepaste ontwerpprincipes	16
4.2. Authenticatie, autorisatie en security	17
4.2.1. JWT-tokenopslag en XSS	17
4.2.2. Accountvergrendeling	17
5. DATABASE	18
5.1. Tabelstructuur	18
5.2. Null-regels (NA / NNA)	18
5.3. Verwijderingsregels (ODC / ODN / ODR)	18
5.4. Relaties tussen tabellen	19
6. COMMUNICATIE MET CLIËNTEN	20
6.1. Use Cases	20
6.2. Moscow-prioritering	20
6.3. Gebruikerhandleiding	22
6.3.1. Handleiding Patiëntenpagina	22
6.4. Meetings en Mails	22
6.5. Onderzoeksvragen	23

6.5.1. Waarom wordt elke dataflowrecord opgeslagen als een aparte record, en niet in een array zoals bij MatLab?	23
7. FLOW DIAGRAMS	24
8. PLANNING	25
9. BESLUIT	27
9.1. Conclusie	27
9.2. Evaluatie	27
9.3. Toekomst	28
LIJST GERAADPLEEGDE WERKEN	29
FIGURENLIJST	31
TABELLENLIJST	31
BIJLAGEN	32
VERKLARENDE BEGRIPPEN	33
VERANTWOORDING GEBRUIK VAN AI	34

1. Inleiding

Voor mijn afstudeerstage van 2026 ben ik gaan solliciteren bij Mobilab & Care. Mobilab & Care is een onderzoeksgroep van Thomas More, verbonden aan het expertisecentrum Zorg en Welzijn. De groep voert praktijk- en wetenschappelijk onderzoek uit op het snijvlak van technologie en zorg. Dankzij het multidisciplinaire experten zoals ingenieurs, therapeuten en sociologen, staan zij bij voor de inclusie van mensen met een beperking. Hun missie is het vertalen van zorgtechnologische innovatie naar een concrete maatschappelijke en economische impact. (Thomas More, z.d.)

Mobilab & Care vroeg mij een longrevalidatiedashboard te ontwikkelen. Voorheen was de patiëntendata van hun intern onderzoeksteam verspreid over uiteenlopende bestandstypes en locaties. Het dashboard moest dit centraliseren: therapeuten, verantwoordelijken en beheerders moesten patiënten en hun bijhorende meetdata overzichtelijk kunnen beheren, opladen en visualiseren.

Dit document beschrijft hoe dat stageproject technisch is gerealiseerd. Het is bedoeld voor zowel vakdeskundigen als lezers zonder IT-achtergrond. Jargon wordt telkens uitgelegd bij het eerste gebruik. Waar nodig wordt verwezen naar gedetailleerde technische bijlagen, die als deliverable voor ontwikkelaars zijn opgesteld.

De opbouw is als volgt. Een analyseluik beschrijft de context en de scope van het project, en verdedigt de architecturale keuzes. Daarna volgt een technische toelichting van de webapplicatie. Architectuur is geen exacte wetenschap: dezelfde applicatie kan door twee verschillende ontwikkelaars op een totaal andere manier worden opgebouwd, afhankelijk van de individuele afwegingen die worden gemaakt en de prioriteiten die worden gesteld. De keuzes die in dit document beschreven worden, zijn dan ook telkens gemotiveerd vanuit concrete overwegingen. Architectuur is in die zin een persoonlijk gegeven: het weerspiegelt hoe een ontwikkelaar naar een probleem kijkt en denkt. Wat voor een schilder een canvas is, is voor een developer een codebase, een lege structuur die pas betekenis krijgt door de beslissingen die erin worden gemaakt. Vervolgens komen de klantgerichte aspecten aan bod: use cases, MoSCoW-prioritering en gebruikersflows. Het document sluit af met een conclusie, een zelfevaluatie en aanbevelingen voor toekomstige ontwikkeling.

Wie een dieper technisch inzicht wenst, kan terecht in de bijlagen: *2-Technical_Documentation_Frontend*, *3-Technical_Documentation_Backend* en *11-Research*.

2. Analyse

2.1. Context

Vooraleer een applicatie wordt gebouwd, is het cruciaal te begrijpen **wie** de applicatie gebruikt, **wat** ze nodig hebben, **waarom**, **hoe** en **wanneer**. Dit werd in een eerste paar ontmoetingen met mijn mentoren Kris Cuppens en Mario Broeckx besproken.

Het onderzoeksteam bestaat uit een tiental medewerkers die de applicatie sporadisch gebruiken, doorgaans één à twee uur per dag. De applicatie moet functioneren op eigen hardware, zoals een laptop of pc, en hoeft nóg niet publiek toegankelijk te zijn via het internet. Dit heeft een grote invloed op de architecturale keuzes, zoals verder wordt toegelicht.

2.2. Scope

Op basis van die eerste meetings werd de **scope** bepaald. Scope is het vakjargon voor de omvang en grenzen van een project: wat zit er in, wat niet.

De scope van het longrevalidatiedashboard omvat het volgende: De applicatie moet een tiental gebruikers kunnen ondersteunen. Gebruikers moeten patiënten kunnen beheren en data uit externe bronnen kunnen opladen, zoals CSV-bestanden, tekstbestanden en smartwatchdata. Daarnaast moet de applicatie in staat zijn een AI-model aan te spreken: gebruikers kunnen geluidsopnames opladen, waarna de output gevisualiseerd wordt. Bestanden kunnen omvangrijk zijn, tot zeker 20.000 lijnen per import. De verwachte belasting bedraagt gemiddeld vijf patiënten per dag, met tien uploads per patiënt, ofwel vijftig uploads per dag. Dit houdt de architectuur bewust eenvoudig.

De AI-integratie is een apart onderwerp dat te technisch is om hier volledig te behandelen. Geïnteresseerde lezers verwijs ik naar bijlage:

11-Research, sectie 3: Hoe AI integreren.

2.3. Frontend versus backend: het huismodel

Om de structuur van een webapplicatie begrijpelijk te maken, helpt de volgende analogie.

Een applicatie is als een huis. De **frontend** is de gevel: wat je ziet, aanraakt en ervaart als bezoeker. De **backend** is de binnenkant: de leidingen, de elektriciteit, de kelder met de voorraad, de eigenaar. Jij als gebruiker drukt op een belknop aan de gevel voor een stuk brood (een klik in de interface). Die bel stuurt een verzoek naar binnen aan de buurman (een API-call), de buurman haalt het brood uit de kelder (de backend haalt data op uit de database), en het brood verschijnt aan de deur (de pagina toont de gegevens).

Dit principe geldt voor vrijwel elke moderne webapplicatie: de frontend toont en vangt interacties op, de backend verwerkt en bewaart.

2.5. Keuze van technologieën

2.5.1. Frontend: React

Voor de frontend zijn er drie grote spelers in de markt: Angular, React en Vue. Allemaal met hun eigen sterktes en beperkingen.

Tabel 1 Vergelijking Vue, React, Angular

	Vue	React	Angular
Type	Framework	Bibliotheek	Framework
Uitgave	2014	2013	2010
Bedrijven	Alibaba, Gitlab	Meta, Uber	Google

Angular is een volledig framework, ondersteund door Google, dat geschikt is voor grootschalige bedrijfsapplicaties. Het vereist TypeScript, een uitbreiding op JavaScript die extra typeveiligheid biedt maar ook een steilere leercurve heeft. Voor dit project, dat groot van scope is maar beperkt in complexiteit (voornamelijk beheerschermen), zou Angular meer overhead meebrengen dan nodig. Bovendien bedraagt de stageperiode slechts drie maanden, wat het leren van een volledig nieuw framework bemoeilijkt. (BrowserStack, 2025)

Vue is beginnersvriendelijk en lichtgewicht, maar heeft een beperkter ecosysteem dan React en Angular. Het wordt minder ondersteund door grote bedrijven, wat vragen oproept over de lange-termijn stabiliteit. (BrowserStack, 2025)

React is een open-source JavaScript-bibliotheek ontwikkeld door Meta. Het onderscheidt zich door herbruikbare componenten: elk onderdeel van de interface is een zelfstandige bouwsteen die op meerdere plaatsen in de applicatie kan worden ingezet. React maakt gebruik van een virtuele DOM, waardoor wijzigingen in de interface efficiënt worden doorgevoerd zonder de volledige pagina opnieuw te laden. Dit maakt het bijzonder geschikt voor interactieve dashboards. (BrowserStack, 2025)

Het verschil tussen een **framework** en een **bibliotheek** is hier relevant. Een framework dicteert de structuur van je applicatie: het bepaalt hoe je code is georganiseerd. Een bibliotheek biedt gereedschap aan dat je naar keuze inzet, zonder je te verplichten een bepaalde structuur te volgen. React is een bibliotheek, wat meer vrijheid geeft maar ook meer eigen verantwoordelijkheid voor de ontwikkelaar.

De keuze voor React was het meest logisch: de vertrouwdeheid vanuit eerdere ervaring, de aanwezigheid in het Thomas More-curriculum, en de geschiktheid voor interactieve, datarijke dashboards maar wanneer ontwikkelingspraktijken niet worden gerespecteerd, kan het al snel veranderen in een lelijk monster. (BrowserStack, 2025) (Alam, 2024)

2.5.2. Backend: .NET

Voor de backend werd gekozen voor **.NET**, een opensource ontwikkelplatform van Microsoft. .NET is krachtig, performant en schaalbaar, en sluit aan bij **object-georiënteerd programmeren (OOP)**: een stijl van programmeren waarbij de code wordt opgebouwd uit objecten die elk hun eigen gegevens en functies bezitten. Een object zoals *patiënt* houdt zich enkel bezig met zichzelf en zijn eigen data. Dit maakt de code overzichtelijk en onderhoudbaar. (Microsoft, z.d.) (Wikipedia, 2026)

.NET biedt het webframework **ASP.NET Core**, dat specifiek gericht is op het bouwen van web-API's. Een API (*Application Programming Interface*) is de interface waarlangs de frontend verzoeken stuurt naar de backend. Samen met **Entity Framework Core**, een bibliotheek die databasecommunicatie vereenvoudigt, vormt dit een solide basis voor een datagedreven webapplicatie. (Microsoft, z.d.) (CZappstudio, 2023)

Naast technische overwegingen speelt ook een persoonlijke factor mee. Eerder in de opleiding aan Thomas More werd kennisgemaakt met .NET, en dit project bood een unieke kans om hierin verder te groeien.

3. Frontend Architectuur

Het longrevalidatiedashboard is een Single Page Application (SPA), gebouwd met React 19 en Vite. Een SPA is een webapplicatie die bestaat uit één enkele HTML-pagina: in plaats van bij elke actie een volledig nieuwe pagina te laden, wordt enkel het relevante deel van de interface vervangen. Dit zorgt voor een vlotte en responsieve gebruikerservaring, vergelijkbaar met een desktop-applicatie. (Lawson, 2026)

De interface is volledig in het Nederlands opgesteld en onderscheidt drie gebruikersrollen: beheerder, verantwoordelijke en therapeut, elk met een ander toegangsniveau tot de data.

3.1. Architecturale lagen

Voor dit gedeelte is het aangeraden om 'Figuur 2: Frontend architectuur – schematische weergave' op de volgende pagina erbij te nemen.

De frontend is opgebouwd uit drie lagen: Presentation, Feature en Integration. Dit onderscheid is geen toeval maar een bewuste architectuurkeuze.

De **Presentation-laag** bevat de visuele bouwstenen van de applicatie. Per domein (zoals patiënt, sessie of oefening) is er een aparte map met bijhorende componenten. Een component is een herbruikbaar stuk interface: een knop, een formulier, een kaartje. Zo bestaat er een component *CreatePatientModal.jsx* dat het aanmaakformulier voor een patiënt weergeeft. Het bewust gescheiden houden van deze verantwoordelijkheden is een toepassing van het **Single Responsibility Principle (SRP)**, één van de SOLID-ontwerpprincipes: elke module heeft slechts één reden tot wijziging. (Oloruntoba, 2025)

De **Feature-laag** bevat hooks. Een hook koppelt de gebruikersinteractie aan de onderliggende logica. Wanneer een gebruiker een formulier invult en op 'Aanmaken' drukt, is het de hook die de invoer valideert (bijv. dat een patiënt geen lichaamslengte van 1.200 cm heeft) en vervolgens de Integration-laag aanspreekt.

De **Integration-laag** bevat services. Een service is de brug tussen de hook en de backend: hij stuurt een HTTP-verzoek naar de backend-API, bijvoorbeeld een POST-verzoek naar *CreatePatient* met de ingevulde patiëntgegevens.

Deze gelaagde opbouw is een directe toepassing van het **Separation of Concerns**-principe: elke laag heeft een duidelijk afgebakende taak, waardoor wijzigingen in één laag geen onverwachte gevolgen hebben in een andere. Het resultaat is een codebase die beter beheersbaar, beter testbaar en toegankelijker is voor nieuwe ontwikkelaars. (Tonka, 2023)

Maak een nieuwe patiënt aan

PatiëntID: P25 Gewicht (kg): 90

Height (cm): 1200 Geboortejaar: 1990

Geslacht: Man Medicatie (optioneel): Paracetamol

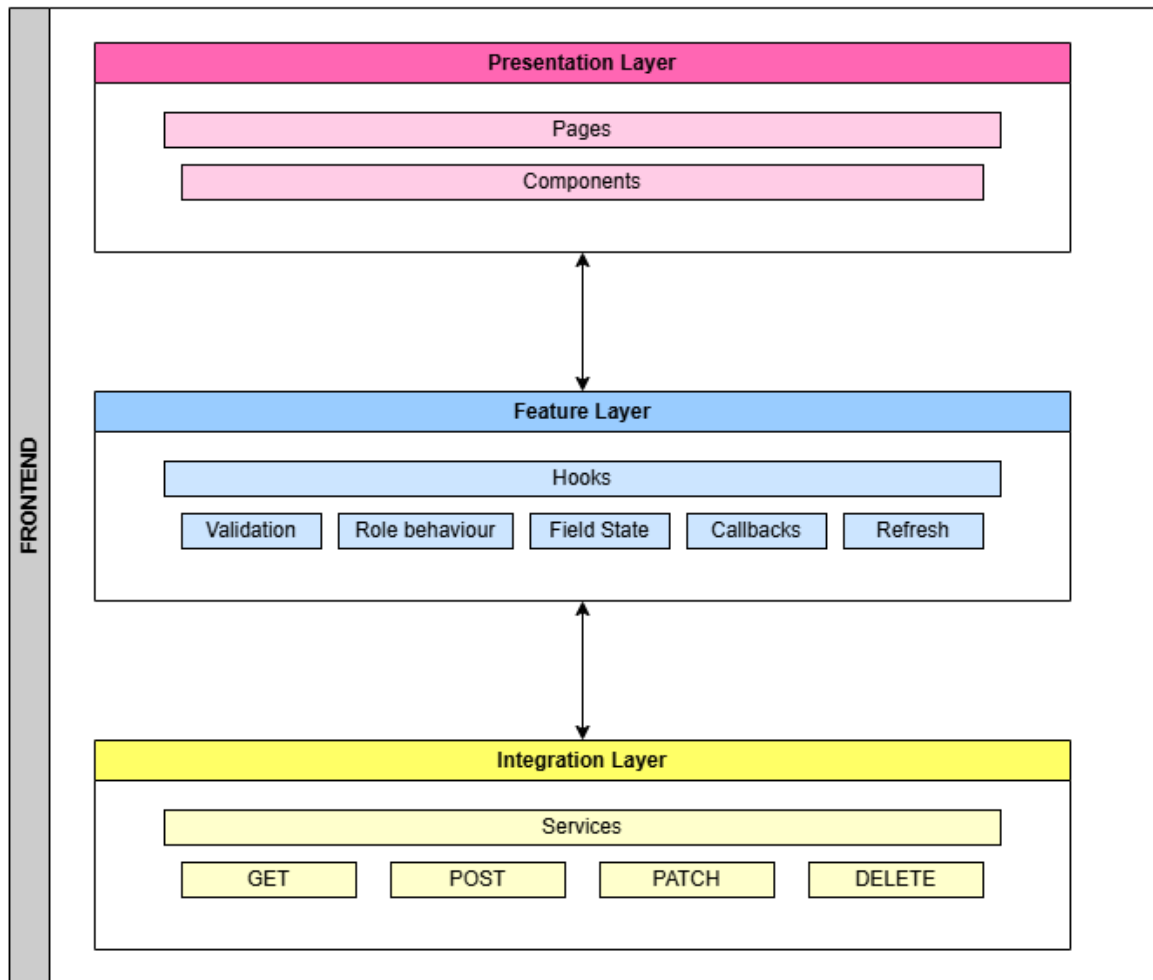
Pathologieën (optioneel):
☒ Asthma ☐ Bronchiectasis
☐ Cystic Fibrosis

☒ Ik bevestig dat de patiënt geïnformeerd is en toestemming heeft gegeven voor de verwerking van zijn/haar gegevens.

Vul een geldige lengte in (50 - 300cm).

Annuleer Aanmaken

Figuur 1 CreatePatientModel.jsx



Figuur 3: Frontend Architectuur – schematische weergave

3.2. Projectstructuur (vereenvoudigd)

Hieronder een vereenvoudigde weergave van de mappenstructuur voor de patiënt-entiteit, waarbij de drie lagen zichtbaar zijn.

```
+--- components/
|   +--- patient/
|   |   |   CreatePatientModal.jsx
|   |   |   PatchPatientModal.jsx
|   |   |   PatientCard.jsx
|   |   |   PatientList.jsx
|   |   \--- hooks/
|   |       useCreatePatient.js
|   |       useDeletePatient.js
|   |       usePatchPatient.js
+--- pages/
|   |   PatientPage.jsx
|   \--- hooks/
|       usePatients.js
\--- services/
     patientService.js
```

3.3. Authenticatie & Autorisatie

In nagenoeg elke applicatie moet worden nagedacht over wie toegang heeft en tot wat. Twee begrippen zijn hier essentieel:

Authenticatie beantwoordt de vraag: *wie ben jij?* Een gebruiker bewijst zijn identiteit, bijvoorbeeld via een e-mailadres en wachtwoord of via Google. Dit is vergelijkbaar met de sleutel die iemand nodig heeft om een huis binnen te komen. (GeeksForGeeks, 2026)

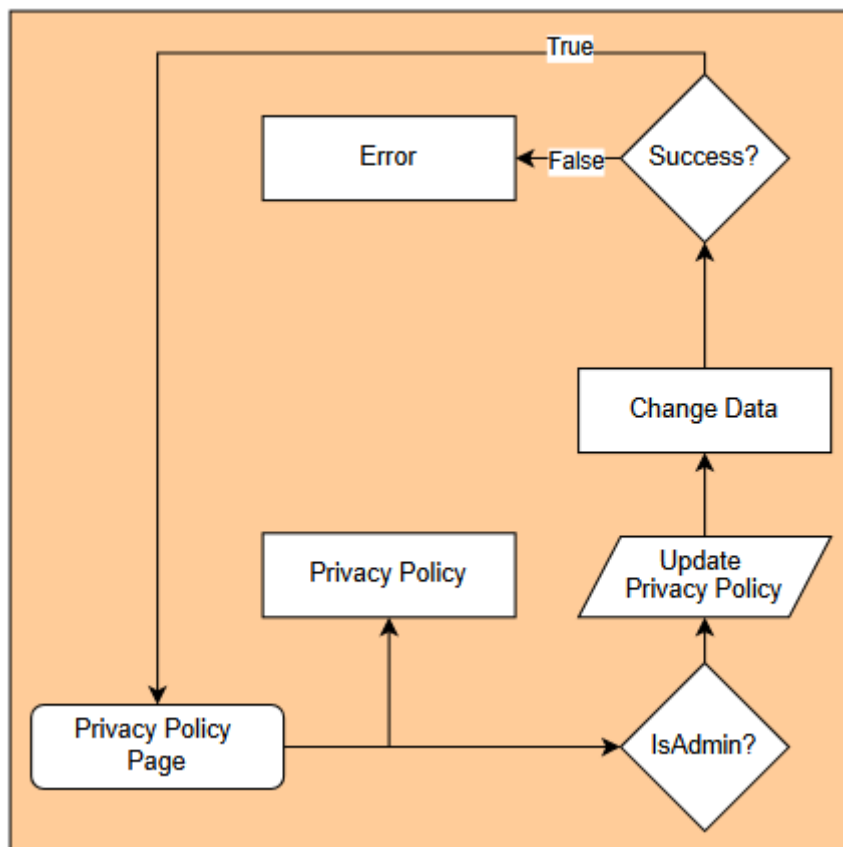
Autorisatie beantwoordt de vraag: *wat mag jij?* Eenmaal binnen bepaalt de rol van de gebruiker welke kamers hij mag betreden. Een therapeut mag zijn eigen patiënten zien, maar niet die van collega's. Een beheerder heeft volledige toegang. (GeeksForGeeks, 2026)

In de applicatie verloopt dit als volgt. Na een succesvolle aanmelding stuurt de backend een JSON Web Token (JWT) terug. Een JWT is een digitaal handtekeningformulier: het bevat de identiteit en de rol van de gebruiker, en is cryptografisch ondertekend door de server zodat het niet vervalst kan worden. Bij elke volgende actie stuurt de frontend dit token mee, waarna de backend verifieert of de gebruiker de gevraagde bewerking mag uitvoeren. (Jwt, z.d.)

Tabel 2 Rollen

Role	Access Scope
Therapeut	Kunnen alleen eigen patiënten beheren.
Verantwoordelijke	Kunnen eigen patiënten en patiënten van therapeuten waarover ze machtiging hebben bekijken.
Administrator	Volledige toegang, kunnen alle patiënten beheren.

Op volgende pagina kan je 'figuur 3: Privacy Policy Flow' terugvinden. Daar zie je een flowdiagram waarbij autorisatie wordt toegepast. Iedereen kan de Privacy Policy bekijken, maar alleen beheerders kunnen het aanpassen.



Figuur 5: Privacy Policy Flow

3.4. Notabel component

3.4.1. ProtectedRoute

Bij elke pagina die een gebruiker bezoekt, wordt het component **ProtectedRoute** uitgevoerd. Dit component fungeert als poortwachter en doorloopt de volgende stappen:

- Is de gebruiker niet ingelogd? Stuur hem naar de loginpagina.
- Is er al gecontroleerd of de gebruiker alle vereiste juridische documenten heeft ondertekend? Zo niet, start die controle en toon nog niets.
- Zijn er nog niet-ondertekende documenten? Toon een ondertekeningsscherm bovenop de pagina.
- Alles in orde: laat de gebruiker toe tot de pagina.

3.5. Codeerconventies

Om de structuur van het project overzichtelijk en netjes te houden, heb ik volgende codeerconventies gehandhaafd.

Tabel 3 Codeerconventies

Object	Conventie naamgeving	Voorbeeld
Componenten	PascalCase	PatientCard.jsx
Paginas	PascalCase + "Page" suffix	PatientPage.jsx
Services	camelCase + "Service" suffix	patientService.js
Hooks	camelCase + "use" prefix	usePatients.js
Configuratie files	camelCase	defaultParams.js
CSS Klassen	Tailwind utilities + mobilab- prefix	bg-mobilab-header

Een meer technische documentatie in het Engels kan je vinden in de bijlage:
2-Technical_Documentation_Frontend.

4. Backend architectuur

Het longrevalidatiedashboard maakt gebruik van een ASP.NET Core 8.0 Web API als backend. De backend fungeert als de server van de applicatie: hij ontvangt verzoeken van de frontend, verifieert ze, verwerkt de data en stuurt een antwoord terug. Alle patiëntgegevens, sessies, oefeningen, longfunctietesten en smartwatchdata worden door de backend beheerd en opgeslagen in een database.

4.1. Architecturale lagen

Voor dit gedeelte is het aangeraden om 'Figuur 3: Backend architectuur – schematische weergave' op de volgende pagina erbij te nemen.

De architectuur van de backend is opgebouwd uit vijf opeenvolgende lagen, elk met een duidelijk afgebakende verantwoordelijkheid. Om dit toegankelijk te maken, wordt doorheen deze sectie de analogie van een restaurant gebruikt, gevolgd door de technische uitleg. De ober, de keuken, het magazijn en de bewaker aan de deur vormen samen het restaurantverhaal dat de vijf lagen verbindt.

4.1.1. Middleware-laag

Voordat een klant het restaurant binnenstapt, passeert hij de ingang. Aan de deur staat een bewaker die controleert of de klant welkom is: heeft hij een reservering? Staat hij niet op de zwarte lijst? Pas wanneer de klant door alle controles is, mag hij binnenkomen. Wie niet door de controles geraakt, wordt vriendelijk maar duidelijk teruggestuurd.

Middleware is een reeks van verwerkingsstappen die elk HTTP-verzoek doorloopt **voordat** het de eigenlijke applicatielogica bereikt. In deze backend zijn vier middleware-componenten geconfigureerd, in volgorde van uitvoering.

De **ExceptionHandlerMiddleware** vangt onverwachte fouten op die elders in de applicatie optreden en vertaalt ze naar een gestandaardiseerd JSON-antwoord. Dit is beveiligingstechnisch relevant: Volgens OWASP's Proactive Controls moeten uitzonderingen centraal worden beheerd, en mogen foutmeldingen aan gebruikers geen gevoelige technische data bevatten. Onafgehandelde fouten kunnen een aanval informatie geven over het gebruikte framework of de databaseversie, wat de eerste fase van een aanval vergemakkelijkt. (Owasp, 2026)

De **ComplianceCheckMiddleware** verifieert bij elk verzoek of de gebruiker de meest recente juridische documenten heeft aanvaard, zoals de privacyverklaring en de verwerkingsovereenkomst.

CORS (Cross-Origin Resource Sharing) bepaalt welke externe domeinen verzoeken mogen sturen naar de API. Dit voorkomt dat willekeurige websites verzoeken kunnen sturen namens een ingelogde gebruiker.

De **Authentication / Authorization-middleware** verwerkt het JWT-token bij elk inkomend verzoek, verifieert de geldigheid ervan en bepaalt de rol van de gebruiker. Pas daarna wordt het verzoek doorgelaten naar de volgende laag.

4.1.2. API-laag (Controller)

De klant is door de ingang en neemt plaats. Een ober komt aan tafel, noteert de bestelling en geeft die door aan de keuken. De ober bereidt het eten zelf niet: hij is het aanspreekpunt voor de klant en de schakel naar de rest van het restaurant. Wanneer het gerecht klaar is, brengt hij het terug naar de juiste tafel.

De API-laag bestaat uit controllers, één per domein. Een controller ontvangt het inkomende HTTP-verzoek van de frontend, controleert of de meegezonden gegevens geldig zijn, en geeft het verzoek door aan de applicatielaag. Hij bevat zelf geen bedrijfslogica.

De gegevens worden uitgewisseld via **DTO's** (Data Transfer Objects): lichte gegevenscontainers die enkel de informatie bevatten die nodig is voor dat specifieke verzoek, zonder interne domeindetails bloot te stellen. *Hierover meer in sectie 4.2.4.*

4.1.3. Applicatielaag (CQRS, Handlers, Services)

In de keuken werken verschillende specialisten. De ene is verantwoordelijk voor het ophalen van koude gerechten uit de koeling (queries), de andere bereidt warme gerechten of verwijdt overschot (commands). Een keukenchef coördineert wie wat doet en roept de juiste specialist op. Niemand in de keuken spreekt rechtstreeks met de klant.

Dit is de kern van de applicatie. Hier wordt het **CQRS-patroon** (Command Query Responsibility Segregation) toegepast. CQRS scheidt lees- en schrijfoperaties strikt van elkaar: *queries* vragen data op zonder iets te wijzigen, *commands* voeren wijzigingen door zoals aanmaken, bijwerken of verwijderen. Elke query en command heeft een bijhorende **handler**: een klasse die precies één bewerking uitvoert, conform het Single Responsibility Principe. De koppeling tussen controller en handler verloopt via het **Mediator-patroon**: de controller weet niet welke handler zijn verzoek verwerkt. Hij geeft het verzoek door aan de Mediator, die de juiste handler opzoekt en aanroept. Dit bevordert een losgekoppelde architectuur waarbij componenten onafhankelijk van elkaar aangepast of getest kunnen worden. (Code Maze, 2024) (Okenyi, 2024)

Naast queries en commands bevat de applicatielaag ook services voor domeinoverstijgende bedrijfslogica, zoals JWT-verwerking, signaalverwerking en e-mailverzending.

Stel je voor dat het restaurant een vaste messenslijper heeft. De keuken vraagt via een standaardformulier om messen te slijpen, maar het maakt de keuken niet uit op welke manier. Zolang het mes maar scherp is. Morgen kan de restauranthouder een andere messenslijper inschakelen zonder dat de keuken iets hoeft aan te passen.

Om dit principe toe te passen worden services opgesplitst in een **interface** en een concrete implementatie. Een interface is een contract: het beschrijft wat een service moet kunnen doen, zonder te bepalen hoe. De rest van de applicatie werkt uitsluitend met het contract, nooit rechtstreeks met de implementatie.

Dit biedt drie concrete voordelen. Ten eerste maakt het de code consistent: interfaces verplichten jou een structuur aan te houden in je code. Ten tweede is de implementatie flexibel: wanneer een externe service vervangen moet worden, volstaat het een nieuwe implementatie van de interface te schrijven. Ten derde verhoogt het de leesbaarheid: een interface legt in één oogopslag vast wat een service kan, zonder de volledige implementatie te hoeven lezen. (van der Aalsvoort, 2024)

Dit is een toepassing van het **Dependency Inversion Principle** (het vijfde SOLID-principe): hogere lagen hangen af van abstracties, niet van concrete implementaties.

4.1.4. Domeinlaag

Tussen de keuken en het magazijn bevindt zich een vertaalkamer. De keuken werkt met recepten in het intern keukenformaat. Het magazijn slaat alles op in een ander formaat: schappen, codes, gewichten. Een medewerker zet bij het opslaan de keukenopdracht om naar het magazijnformaat, en omgekeerd bij het ophalen.

De domeinlaag bevat de entiteitsklassen (de directe representatie van de databasetabellen) en de bijhorende DTO's. Omdat de interne structuur van een entiteit niet altijd overeenkomt met wat de frontend nodig heeft, worden beide via een Mapper omgezet.

Het onderstaande voorbeeld (Tabel illustreert het nut van een DTO. Als de frontend enkel het adres van een patiënt nodig heeft, stuurt de backend niet de volledige entiteit terug, maar enkel een DTO met de relevante velden. Dit beschermt de interne datastructuur en vermindert onnodig dataverkeer.

De domeinlaag bevat de entiteitsklassen, de directe representatie van de databasetabellen, en de bijhorende Data Transfer Objects (DTO's). Omdat de interne structuur van een entiteit niet altijd overeenkomt met wat de frontend nodig heeft, en omdat het onwenselijk is om interne databasestructuren rechtstreeks bloot te stellen, worden beide via een *Mapper* omgezet. Bij een schrijfoperatie zet de mapper een binnenkomende RequestDTO om naar een entiteit die kan worden opgeslagen. Bij een leesoperatie zet hij een entiteit om naar een ResponseDTO die wordt teruggestuurd. Dit beschermt de interne domeinstructuur en maakt het mogelijk beide onafhankelijk van elkaar te wijzigen. Een voorbeeld van een DTO waarbij wij enkel het adres van een persoon willen terugkrijgen.

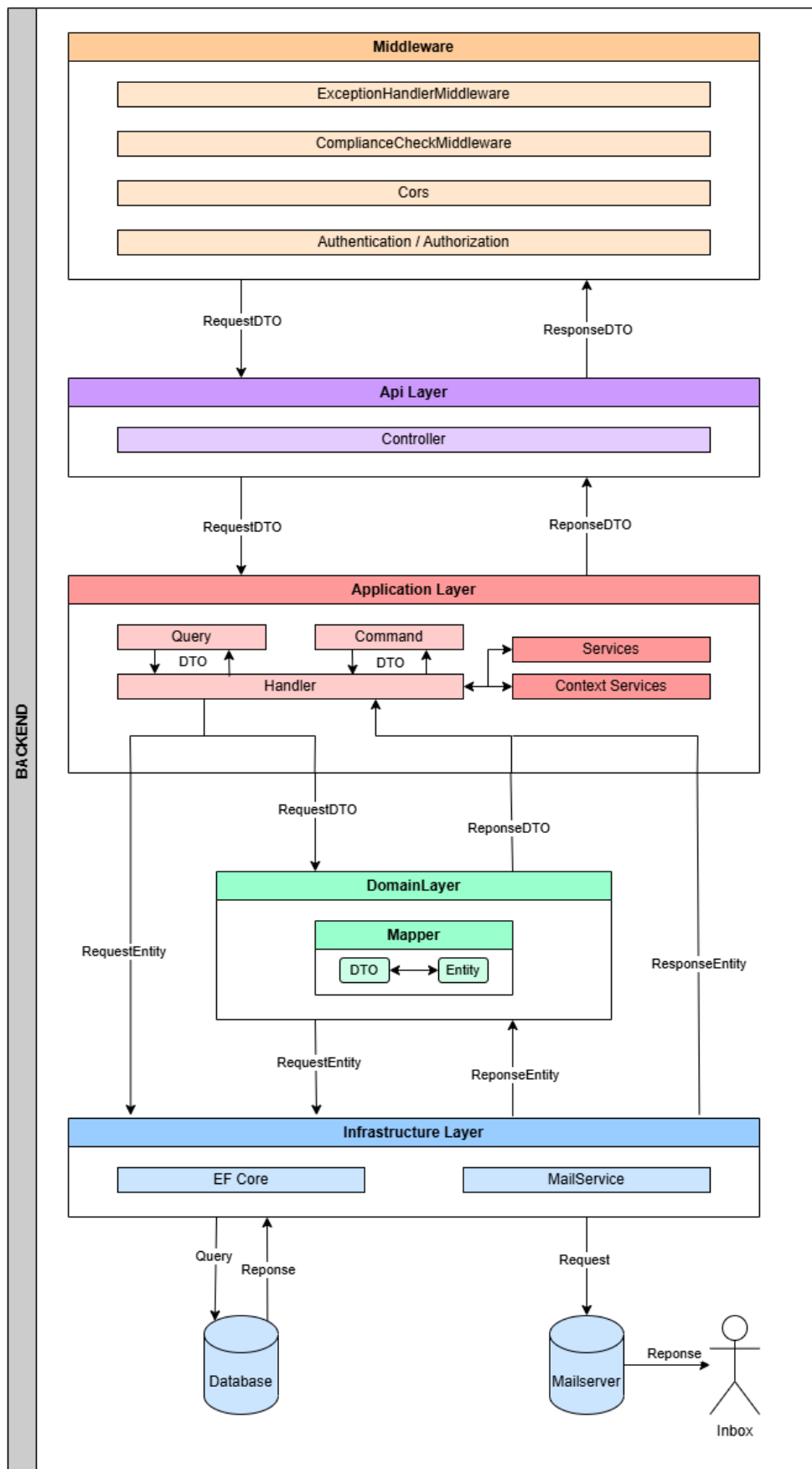
Tabel 4: Entiteit versus DTO

Entiteit (volledige data)	DTO (enkel wat nodig is)
Naam: Jenny Tutone Straat: Springstraat Nummer: 21 Locatie: Geel Telefoon: 867-5309	Naam: Jenny Tutone Adres: Springstraat 21, Geel

4.1.5. Infrastructuurlaag

Het magazijn is de laatste schakel. Het beheert de voorraad en voert opdrachten uit die het van de keuken ontvangt. Een tweede afdeling verzorgt verzendingen naar buiten: wanneer een klant een bevestigingsmail moet ontvangen, vertrekt die vanuit dit magazijn.

De infrastructuurlaag verzorgt alle communicatie met externe systemen. **Entity Framework Core (EF Core)** vertaalt entiteitsklassen automatisch naar databaseoperaties, zodat geen handmatige SQL nodig is. De **AppDbContext** is de centrale toegangspoort tot de database. De **MailService** verzendt e-mails via een externe mailserver, bijvoorbeeld voor accountactivatie of wachtwoordherstel.



Figuur 6: Backend architectuur – schematische weergave

4.1.6. Projectstructuur (vereenvoudigd)

Vereenvoudigde weergave van de projectstructuur van enkel de patiënt-entiteit, waarbij je de meerdere lagen kan terugvinden.

```
+--- Controllers/                ← API-laag
|    PatientController.cs
+--- Features/                  ← Applicatielaag
|    +--- Patients/
|    |    +--- Commands/
|    |    |    CreatePatientCommand.cs
|    |    |    CreatePatientHandler.cs
|    |    \--- Queries/
|    |         GetPatientQuery.cs
|    |         GetPatientHandler.cs
+--- Entities/                 ← Domeinlaag
|    +--- Domain/
|    |    PatientEntity.cs
|    +--- Dto/Patient/
|    |    PatientDTO.cs
|    \--- Mappers/
|         PatientMapper.cs
+--- Services/
|    +--- Authorization/
|         IPatientAccessService.cs
|         PatientAccessService.cs
\--- DataAccess/               ← Infrastructuurlaag
     AppDbContext.cs
```

4.1.7. Toegepaste ontwerpprincipes

De gelaagde architectuur is geen toevallige keuze, maar een bewuste toepassing van het **Clean Architecture**-patroon, beschreven door Robert C. Martin. De kern is de *Dependency Rule*: broncode-afhankelijkheden mogen uitsluitend naar binnen wijzen. Niets in een binnenste laag mag iets weten over een buitenste laag. In de praktijk: de domeinlaag kent EF Core niet, en de applicatielaag kent de controller niet. Wijzigingen in de infrastructuur hebben daardoor geen invloed op de bedrijfslogica. (Clean Coder, 2012)

Binnen elke laag wordt het **Separation of Concerns**-principe toegepast: elke klasse heeft één afgebakende verantwoordelijkheid. Dit is een directe toepassing van het **Single Responsibility Principle**, het eerste principe van SOLID. Wijzigingen in één component hebben daardoor minimale gevolgen voor de rest van de applicatie. (DigitalOcean, 2025)

Het scheiden van services in Interfaces en services is een toepassing van het **Dependency Inversion Principle** en het **Interface Segregation Principle**, interfaces zijn gericht en beperkt tot wat een klasse effectief nodig heeft, resulteert dit in een architectuur waarbij componenten onafhankelijk van elkaar kunnen evolueren. (Clean Coder, 2012) (Code Maze, 2024) (DigitalOcean, 2025)

4.2. Authenticatie, autorisatie en security

De authenticatie- en autorisatielogica in de backend sluit aan bij wat eerder in sectie 3.3 werd beschreven. Hieronder worden de beveiligingsmaatregelen op serverniveau verder toegelicht.

4.2.1. JWT-tokenopslag en XSS

De JSON Web Tokens worden in de frontend opgeslagen via **sessionStorage**. Dit kan worden vergeleken met een huissleutel die enkel bestaat zolang het tabblad open is: zodra de gebruiker het tabblad sluit, verdwijnt de sleutel automatisch. Dit staat in contrast met *localStorage*, waarbij de sleutel bewaard blijft ook nadat de gebruiker het venster sluit.

SessionStorage is om die reden veiliger dan localStorage. Toch is het niet zonder risico: volgens de OWASP HTML5 Security Cheat Sheet is sessionStorage toegankelijk via JavaScript, waardoor een **Cross-Site Scripting (XSS)**-aanval in theorie het token kan stelen. XSS is een aanvalstechniek waarbij een aanvalleur kwaadaardige Javascript-code in een webpagina injecteert. OWASP beveelt httpOnly-cookies aan als veiligere alternatief, omdat die ontoegankelijk zijn voor JavaScript. (Owasp, z.d.)

Deze trade-off is bewust gemaakt. De keuze voor sessionStorage hangt samen met het feit dat **httpOnly**-cookies nog niet zijn geïmplementeerd. De applicatie is ontworpen om lokaal te draaien binnen een gecontroleerde klinische omgeving, zonder publieke blootstelling aan het internet. In die context vormt het XSS-risico geen realistische bedreiging. Er is op de frontend op dit moment geen aanvalfront beschikbaar waarlangs een dergelijke aanval zou kunnen plaatsvinden. In de technische documentatie wordt desondanks aanbevolen om de loginflow in een toekomstige versie te beveiligen via een gespecialiseerde authenticatiedienst zoals Auth0, wat ook de implementatie van httpOnly-cookies overbodig maakt. (Owasp, z.d.)

4.2.2. Accountvergrendeling

Na drie opeenvolgende mislukte inlogpogingen wordt een account gedurende vijftien minuten vergrendeld. Dit stemt overeen met de aanbevelingen van de OWASP Web Security Testing Guide (WSTG-ATHN-03), die stelt dat accounts doorgaans worden vergrendeld na drie tot vijf mislukte pogingen en pas na een vooraf bepaalde periode worden ontgrendeld. (Owasp, z.d.)

Daarnaast valideert de backend elk JWT volledig: de uitgever (issuer), het publiek (audience), de levensduur (lifetime) en de ondertekeningssleutel (signing key) worden geverifieerd. De OWASP API Security Top 10 identificeert het ontbreken van deze validaties expliciet als een kwetsbaarheid onder Broken Authentication. (Owasp, z.d.)

5. Database

De database is een **SQL Server**-relationele database en is gemodelleerd via een ERD-diagram (Entity Relationship Diagram). Een ERD is een visuele voorstelling van de structuur van de database: welke tabellen er zijn, welke gegevens ze bevatten en hoe ze zich tot elkaar verhouden.

Hieronder wordt de relatie tussen de tabellen Patiënt en Sessie als voorbeeld toegelicht.

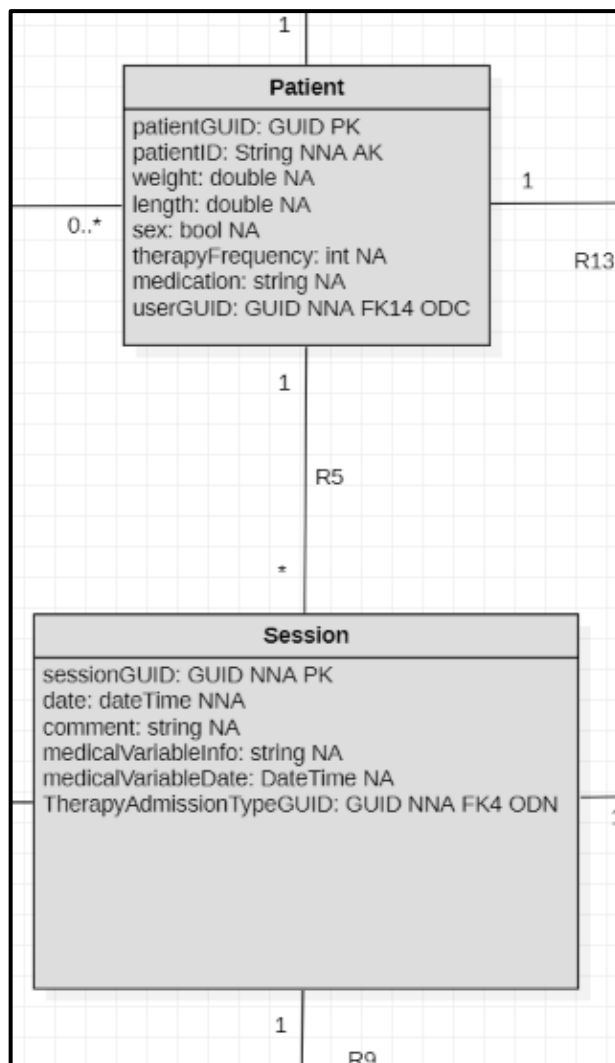
5.1. Tabelstructuur

De tabel Patiënt bevat de volgende velden:

- **ID:** Een unieke identifier die automatisch door het systeem wordt toegekend.
- **Patiënt-ID:** Een leesbaar identificatienummer dat door een gebruiker wordt toegewezen.
- **Gewicht, lengte en geslacht:** Medische basisgegevens van de patiënt.
- **TherapyFrequency:** Het aantal sessies dat de patiënt reeds heeft gehad.
- **Medication:** Eventuele medicatie.
- **UserGUID:** Een verwijzing naar de gebruiker waartoe deze patiënt behoort.

5.2. Null-regels (NA / NNA)

Bij elk veld is aangegeven of het verplicht ingevuld moet worden. **NA** (Nulls Allowed) betekent dat het veld leeg mag blijven bij het aanmaken of aanpassen van een record. **NNA** (Nulls Not Allowed) betekent dat het veld verplicht is.



Figuur 7: ERD – relatie Patiënt en Sessie

5.3. Verwijderingsregels (ODC / ODN / ODR)

Wanneer een gebruiker of patiënt uit het systeem wordt verwijderd, bepalen verwijderingsregels wat er met de gekoppelde data moet gebeuren. Er zijn drie soorten:

- **ODN – On Delete Null:** Als de gekoppelde record wordt verwijderd, wordt dit veld op leeg (null) gezet.
- **ODR – On Delete Restrict:** Een record kan niet worden verwijderd zolang er gekoppelde records bestaan.
- **ODC – On Delete Cascade:** Als een record wordt verwijderd, worden alle gekoppelde records ook automatisch verwijderd.

In deze applicatie wordt **ODC** toegepast op de relatie tussen gebruikers en patiënten, en tussen patiënten en hun data. Dit is een bewuste keuze vanuit GDPR-perspectief: artikel 17 van de GDPR (het recht op verwijdering, ook wel het 'recht om vergeten te worden' genoemd) vereist dat persoonsgegevens op verzoek volledig en permanent verwijderd kunnen worden. Door cascade-verwijdering te gebruiken, worden bij het

verwijderen van een gebruiker of patiënt automatisch alle bijhorende gegevens mee verwijderd, zonder dat dit manueel beheerd hoeft te worden. (GDPR, art. 17 'right to be forgotten')

5.4. Relaties tussen tabellen

Tussen de tabellen Patiënt en Sessie bestaat een **1-op-veel-relatie (0..*)**: een patiënt kan nul of meerdere sessies hebben, maar elke sessie behoort altijd tot precies één patiënt. Dit is de standaardmanier om hiërarchische gegevens te structureren in een relationele database.

Een volledig overzicht van alle tabellen en hun onderlinge relaties is te vinden in de bijlage: UML_Model.

6. Communicatie met cliënten

Een applicatie bouwen zonder voortdurend af te stemmen met de opdrachtgever is een recept voor problemen. Functies die gebouwd worden op basis van aannames komen zelden overeen met wat de cliënt effectief nodig heeft. Om dit te vermijden, werden de eerste vijf weken wekelijkse demomeetings georganiseerd waarbij de vereisten grondig in kaart werden gebracht.

6.1. Use Cases

Om de vereisten van de applicatie op een gestructureerde manier te documenteren, werden use cases opgesteld. Een use case is een methodologie uit systeemanalyse die beschrijft wie een bepaalde functie kan uitvoeren, onder welke voorwaarden en hoe het systeem reageert, zowel bij een normale stroom als bij afwijkingen.

Elke use case bestaat uit de volgende onderdelen:

- **Functionality:** Wie kan wat uitvoeren?
- **Precondition:** Aan welke voorwaarden moet voldaan zijn?
- **Normal Flow:** Hoe verloopt de functie stap voor stap?
- **Alternatives:** Wat gebeurt er als er iets fout loopt?

5.3.UC-PAT-3: Patiënt aanmaken

Functionality: Als een therapeut, kan ik een nieuwe patiënt aanmaken.

Precondition: Gebruiker is ingelogd.

Normal Flow: Gebruiker vult patiënt-ID, gewicht, lengte, geslacht en geboortjaar en vinkt aan dat hij/zij de patiënt hierover heeft geïnformeerd en drukt op 'Aanmaken'. Systeem slaat de nieuwe patiënt op en toont een success-melding.

Alternatives:

Patiënt-ID niet ingevuld → errormessage: "Patient ID is required."

Patiënt-ID langer dan 100 tekens → errormessage: "Patient ID must be under 100 characters."

Gewicht buiten bereik (0-500) → errormessage: "Weight must be between 0 and 500."

Lengte buiten bereik (50-300) → errormessage: "Length must be between 50 and 300."

Geslacht niet ingevuld → errormessage: "Sex is required."

Geboortjaar buiten bereik (1900-2100) → errormessage: "Birth year must be between 1900 and 2100."

Figuur 8: Use Case - Patiënt aanmaken

*Een volledige lijst van alle Use Cases kan je terugvinden in de bijlage:
5-Use_Cases*

6.2. Moscow-prioritering

Niet alle Use Cases zijn even urgent. Om te bepalen welke functies als eerste gebouwd moesten worden, werd het **MoSCoW-principe** toegepast. Dit is een prioriteringstechniek waarbij elke functie in één van vier categorieën wordt geplaatst:

- **Must Have:** Absoluut noodzakelijk. Zonder deze functies is de applicatie niet bruikbaar.
- **Should Have:** Belangrijk, maar niet kritisch voor de eerste oplevering.
- **Could Have:** Meerwaarde, maar enkel te implementeren als er tijd over is.
- **Won't Have:** Bewust buiten scope geplaatst voor dit project.

Tabel 5 MoSCow

Must Have	Should Have	Could Have	Won't Have
UC-AUTH-1	UC-AUTH-2	UC-GEB-6	UC-LOM-5
UC-LEG-1	UC-AUTH-3	UC-PAT-7	UC-ROL-1
UC-LEG-2	UC-PRO-1	UC-OEF-6	UC-ROL-2
UC-LEG-3	UC-PRO-2	UC-TOT-1	UC-ROL-3
UC-LEG-4	UC-PRO-3	UC-TOT-2	UC-ROL-4
UC-GEB-1	UC-GEB-4	UC-TOT-3	
UC-GEB-2	UC-LOM-1	UC-TOT-4	
UC-GEB-3	UC-LOM-3	UC-PATH-1	
UC-GEB-5	UC-SWD-1	UC-PATH-2	
UC-PAT-1	UC-SWD-2	UC-PATH-3	
UC-PAT-2	UC-SWD-3	UC-PATH-4	
UC-PAT-3	UC-ERR-1		
UC-PAT-4	UC-ERR-2		
UC-PAT-5	UC-ERR-3		
UC-PAT-6	UC-ERR-4		
UC-SES-1			
UC-SES-2			
UC-SES-3			
UC-SES-4			
UC-SES-5			
UC-SES-6			
UC-OEF-1			
UC-OEF-2			
UC-OEF-3			
UC-OEF-4			
UC-OEF-5			
UC-FLM-1			
UC-FLM-2			
UC-FLM-3			
UC-FLM-4			
UC-PAG-1			
UC-PAG-2			
UC-LOC-1			
UC-LOC-2			
UC-LOC-3			
UC-LOC-4			

Een belangrijk inzicht dat ik hier uit meeneem: cliënten weten vaak zelf nog niet precies wat ze willen. Daardoor kan de scope van een project snel groeien, of kunnen features na weken ontwikkeling plots niet meer relevant zijn. Het is de verantwoordelijkheid van de developer om duidelijke grenzen te stellen: wat is mogelijk, wat niet, en vanaf wanneer kunnen requirements niet meer ingrijpend wijzigen. Dit principe heb ik consequent toegepast, wat de efficiëntie van het project sterk ten goede is gekomen. (Agile Business, z.d.)

6.3. Gebruikerhandleiding

Naast de technische documentatie voor ontwikkelaars zijn er ook gebruikershandleidingen opgesteld voor de eindgebruikers. Voor elke pagina van de applicatie bestaat een afzonderlijke handleiding. In totaal zijn er acht. Hieronder een voorbeeld voor de patiëntenpagina.

6.3.1. Handleiding Patiëntenpagina

Doel: Op deze pagina kunnen patiënten worden aangemaakt, aangepast, verwijderd en opgevolgd.

Toegang en rechten:

Tabel 6: Toegang per rol - patiëntenpagina

Actie	Beheerder	Verantwoordelijke	Therapeut
Patiënt aanmaken	X	X	X
Patiënt aanpassen	X	X	X
Patiënt verwijderen	X	X	X
Sessies bekijken	X	X	X

Patiënt aanmaken

Tabel 7 Patiënt Aanmaken

Actie	Beschrijving
Druk op de knop "Patiënt aanmaken"	Een aanmaakmodal gaat open
Vul patientgegevens in	
Check de "patiënt geïnformeerd" checkbox	
Druk op "Aanmaken"	

Patiënt verwijderen

Als een patiënt wordt verwijderd, word ook alle patiëntdata verwijderd.

Tabel 8 Patiënt verwijderen

Actie	Beschrijving
Op de patiëntkaart, druk op de knop "Patiënt verwijderen"	Een deletemodal gaat open "Ben je zeker dat je patiënt x wil verwijderen?"
Druk op "Verwijderen"	

Veelgemaakte fouten

Tabel 9 Veelgemaakte fouten

Probleem	Oplossing
Niet de juiste pathologieën beschikbaar	Volg de instructies in "Handleiding_pathologieën"

6.4. Meetings en Mails

Naast de wekelijkse demomeetings werden ook andere overlegmomenten georganiseerd. Zo vond er een meeting plaats met een KU Leuven-student en zijn begeleider over de integratie van een AI-model. De voorbereiding hiervoor is te vinden in bijlage 11-Research, sectie 3.

Een praktisch aandachtspunt dat tijdens het project opdook: de bibliotheek **MediatR**, gebruikt voor het Mediator-patroon in de backend, is in 2025 van licentiemodel veranderd en is gedeeltelijk commercieel

geworden. Na een mail met de CEO van Lucky Penny Software, de developer achter MediatR, bleek het project niet binnen de gratis Community License te vallen.

De mail kan je terugvinden in de bijlage:

15-Mails -mail-Lucky_Penny_Software

6.5. Onderzoeksvragen

Wanneer tijdens een meeting een vraag naar boven kwam waarop geen onmiddellijk antwoord beschikbaar was, werd die behandeld als een onderzoeksvraag. De bevindingen werden nadien schriftelijk onderbouwd. Hieronder een voorbeeld.

6.5.1. Waarom wordt elke dataflowrecord opgeslagen als een aparte record, en niet in een array zoals bij MatLab?

Mag dit in SQL?

SQL Server is ontworpen om grote hoeveelheden gestructureerde gegevens op een betrouwbare manier op te slaan, op te vragen en te koppelen aan andere tabellen (Een oefening heeft meerdere flowrecords, een flowrecords behoren tot één oefening). In een webapplicatie is dat belangrijk, omdat de data niet alleen bewaard moet blijven, maar ook op een flexibele manier gebruikt moet kunnen worden. Je wil bijvoorbeeld metingen per oefening kunnen ophalen, sorteren op tijd, minima en maxima berekenen, of later analyses toevoegen. Dat gaat veel eenvoudiger wanneer elke meting een eigen record is

Het alternatief, namelijk alle meetwaarden als één array of blob opslaan, zou technisch wel kunnen, maar zou niet passen bij het concept van een relationele database. In dat geval gebruik je SQL Server eerder als opslagplaats voor één groot gegevenspakket, in plaats van als databank die de data relationeel kan structureren en efficiënt kan bevragen. Voor een webapplicatie is dat minder handig, omdat je later vaak nog wil filteren, vergelijken, valideren of uitbreiden. Het is dus volledig normaal om elke afzonderlijke meting als een aparte record in de database op te slaan, ook als één oefening 10.000 tot 15.000 datapunten bevat. Zelfs al zijn het er 150.000.

Verschil MatLab en SQL

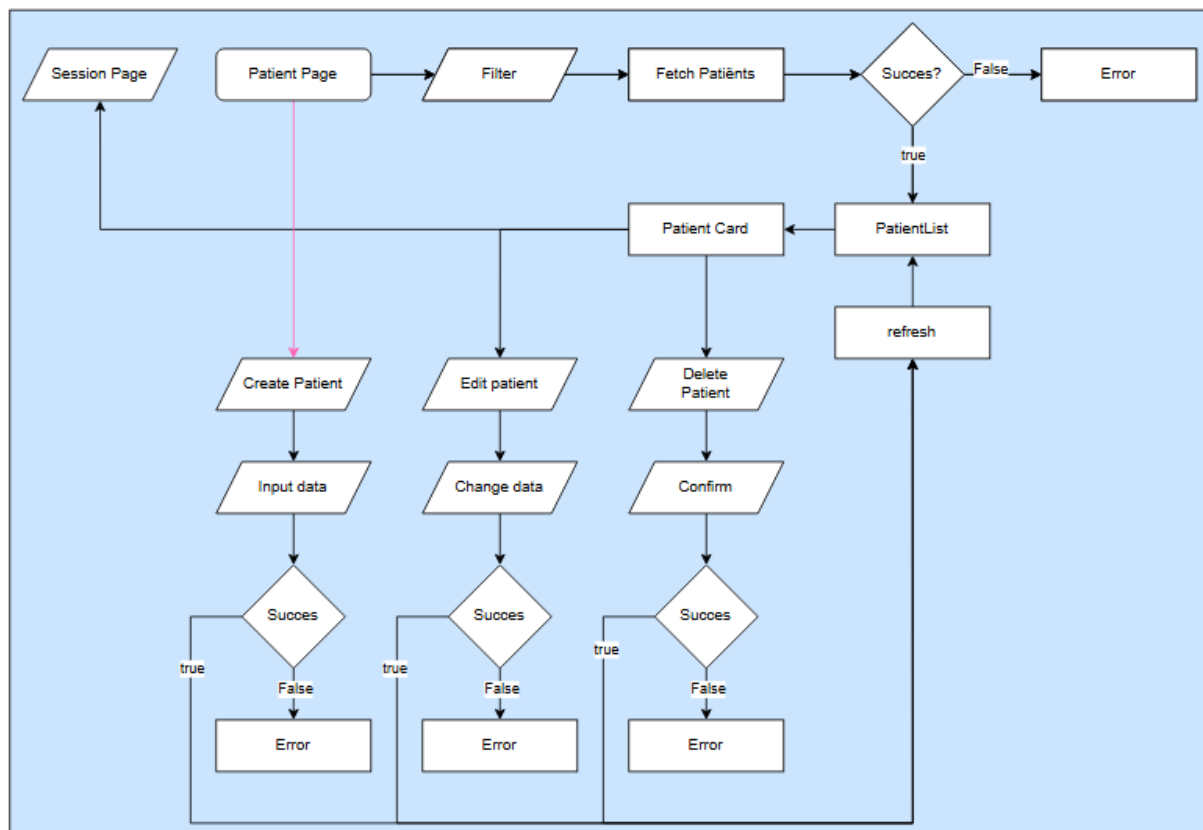
MATLAB en SQL Server hebben een totaal ander doel. MATLAB is gebouwd voor numerieke berekeningen, signaalverwerking en werken met arrays en matrices in het geheugen. Daar is het logisch om een volledige meting als één array te behandelen, omdat je die vooral wil analyseren en wiskundig verwerken. SQL Server daarentegen is gemaakt voor opslag, structuur, relaties tussen gegevens, transacties en efficiënte queries. Het is niet ontworpen als rekentool voor signalen, maar als systeem om gegevens duurzaam te bewaren en later gericht op te halen.

Voor een webapplicatie betekent dat meestal het volgende: de ruwe meetpunten worden als afzonderlijke rijen in SQL Server opgeslagen, daarna haalt de .NET-back-end die gegevens op en zet ze om naar lijsten of arrays om er berekeningen op uit te voeren. Met andere woorden: SQL dient als opslaglaag, terwijl de verwerking gebeurt in de applicatielaag.

7. Flow diagrams

Wanneer alle use cases vastliggen, biedt een **flowdiagram** een visueel overzicht van hoe een gebruiker door de applicatie beweegt. Een flowdiagram toont de volgorde van schermen, beslissingen en acties op een overzichtelijke manier.

Hieronder een voorbeeld voor de patiëntenpagina. Een gebruiker kan via deze pagina een patiënt aanmaken, die vervolgens als kaart in de lijst verschijnt. Via die kaart kan de gebruiker de patiënt verwijderen, aanpassen, of doorklikken naar de sessiepagina.



Figuur 9: Flow Diagram – Patiëntenpagina

Voor elk domein is een afzonderlijk flowdiagram beschikbaar, alsook een overzicht van de volledige applicatieflow. Deze zijn te vinden in bijlage *9-Flowchart*.

8. Planning

De planning is opgebouwd op basis van de use cases. Elke week bevat een overzicht van de use cases die gepland stonden, aangevuld met taken die buiten de use cases vallen, zoals documentatie, refactoring en technische opzet. In de loop van het project verschuift de focus geleidelijk van functionaliteit naar documentatie en afwerking.

Tabel 10 Planning

Week 23/02:	Week 02/03:	Week 09/03:	Week 16/03:
Use cases	Use cases	Use cases	Use cases
UC-GEB-1 UC-PAT-1 UC-PAT-2 UC-SES-1 UC-OEF-1 UC-PAG-1 UC-PAG-2	UC-GEB-2 UC-LOM-1 UC-LOM-3 UC-GEB-5 UC-PAT-3 UC-PAT-5 UC-SES-3 UC-SES-5 UC-OEF-2 UC-OEF-4 UC-FLM-1 UC-FLM-2 UC-FLM-3 UC-FLM-4	UC-GEB-6 UC-PAT-7 UC-OEF-6 UC-SWD-1 UC-SWD-2 UC-SWD-3 UC-ERR-1 UC-ERR-2 UC-ERR-3 UC-ERR-4 UC-SES-6 UC-LOC-1 UC-LOC-2 UC-LOC-3 UC-LOC-4	UC-TOT-1 UC-TOT-2 UC-TOT-3 UC-TOT-4 UC-PATH-1 UC-PATH-2 UC-PATH-3 UC-PATH-4 UC-GEB-3 UC-PAT-4 UC-PAT-6 UC-SES-2 UC-SES-4 UC-OEF-3 UC-OEF-5
Non-use Case	Non-use Case	Non-use Case	Non-use Case
Database UML	Database UML	Unit Tests	Unit Tests
		Refactor	Database UML
			Restyle
DEMOMEETING	DEMOMEETING	DEMOMEETING	

Week 23/03:	Week 30/03:	Week 06/04:	Week 13/04:
Use cases	Use cases	Use cases	Use cases
UC-AUTH-1 UC-PRO-1 UC-PRO-2 UC-PRO-3	UC-GEB-6 UC-PAT-7 UC-OEF-6 UC-SWD-1 UC-SWD-2 UC-SWD-3 UC-ERR-1 UC-ERR-2 UC-ERR-3 UC-ERR-4	UC-LEG-1 UC-LEG-2 UC-LEG-3 UC-LEG-4	
Non-use Case	Non-use Case	Non-use Case	Non-use Case
Unit Tests	Docker Setup	Docker Setup	
Frontend-documentation	Frontend-documentation	Frontend-documentation	Use-Cases
Backend-Documentation	Backend-Documentation	Backend-Documentation	Flow-Chart
Use-Case Diagram	Restyle	Database chances	Sequence-Diagram
Refactor	QOL	Use-Cases	User Guides
Stagepresentatie		Moscow	Privacy Policy
		Privacy Policy	Terms Of Use
		Terms Of Use	Date Processing Agreement
		Date Processing Agreement	
	DEMOMEETING		DEMOMEETING AI MEETING

Week 20/04:	Week 27/04:	Week 04/05:	Week 11/05:
Use cases	Use cases	Use cases	Use cases
AI		BUFFER	BUFFER
Non-use Case	Non-use Case	Non-use Case	Non-use Case
Frontend-documentation	Realisatiedocument	BUFFER	BUFFER
Backend-Documentation	Portfolio	BUFFER	BUFFER
Realisatiedocument		BUFFER	BUFFER
DEMOMEETING DOCUMENTATIE-MEETING		BUFFER	BUFFER

9. Besluit

9.1. Conclusie

Het longrevalidatiedashboard is een webapplicatie gebouwd in React en .NET, ontworpen voor een team van zorgprofessionals bij Mobilab & Care. De applicatie centraliseert het beheer van patiënten, sessies, oefeningen, longfunctietesten en smartwatchdata, en biedt ondersteuning voor drie gebruikersrollen met elk een afzonderlijk toegangsniveau.

De frontend is opgebouwd in drie lagen (Presentatie, Feature, Integratie) en past het Single Responsibility Principe en Separation of Concerns consequent toe. Componenten zijn modulair en herbruikbaar, hooks isoleren de interactielogica van de visuele laag, en services vormen de brug naar de backend.

De backend hanteert een gelaagde Clean Architecture met vijf lagen: Middleware, API, Applicatie, Domein en Infrastructuur. Het CQRS-patroon scheidt lees- en schrijfoperaties, het Mediator-patroon ontkoppelt controllers van handlers, en het Dependency Inversion Principe zorgt voor testbare en uitwisselbare service-implementaties. De database is een SQL Server, gecontaineriseerd via Docker, wat installatie en toekomstig beheer vereenvoudigt.

Voor de cliënt zijn gebruikershandleidingen, use cases, flowdiagrammen en een MoSCoW-overzicht beschikbaar. Voor ontwikkelaars is er uitgebreide technische documentatie, inclusief architectuurdiagrammen, een database-ERD en een researchdocument.

9.2. Evaluatie

De evaluatie van mijn stagebegeleiders was positief. Een concreet verbeterpunt was dat het voor hen niet altijd duidelijk was waar bepaalde documentatie zich bevond of wat erin stond.

Na reflectie begrijp ik waarom dit zo aanvoelde. Tijdens demomeetings behandelde ik mijn begeleiders bewust als cliënten: geen technisch jargon, simpel houden, vragen stellen en grenzen bewaken. Voor de meetings zelf werkte deze aanpak goed. Buiten de meetings bleef ik echter dezelfde houding aanhouden, waardoor ik nooit expliciet en concreet aanwees waar documentatie te vinden was of wat erin stond. Na het evaluatiegesprek heb ik deze aanpak bijgesteld en dat werd positief ontvangen.

Een tweede reflectie gaat over het verschil tussen een framework en een bibliotheek in de praktijk. De backend is gebouwd met een framework (.NET/ASP.NET Core), de frontend met een bibliotheek (React). In de backend heb ik steeds een duidelijk overzicht over de code: het framework legt structuur op en dwingt consistentie af. In de frontend is dat gevoel minder sterk, ondanks meerdere refactors. React geeft de developer volledige vrijheid, maar dwingt geen vaste structuur af. Dit maakt het moeilijker om het overzicht te bewaren, ook al is de code inhoudelijk stabiel. Dit maakt me gemotiveerd om in de toekomst te werken met een frontend-framework, om te ervaren hoe dat aanvoelt in vergelijking.

9.3. Toekomst

Naar de toekomst toe zijn er een aantal concrete aanbevelingen.

Beveiliging van de authenticatieflow: De JSON Web Tokens worden momenteel opgeslagen in sessionStorage, wat in theorie kwetsbaar is voor XSS-aanvallen (Cross-Site Scripting). Omdat de applicatie lokaal draait in een gecontroleerde omgeving zonder publieke blootstelling, vormt dit op dit moment geen realistische bedreiging. Zodra de applicatie online zou gaan, is het sterk aanbevolen de volledige loginflow te vervangen door een gespecialiseerde authenticatiedienst zoals Auth0. Dit elimineert zowel het XSS-risico als de nood aan een eigen implementatie van JWT-beheer.

Juridische documenten: Voordat de applicatie online wordt geplaatst, is het raadzaam om met een jurist of iemand met domeinkennis een volwaardige Privacy Policy, Terms of Service en Data Processing Agreement op te stellen. Elke gebruiker en patiënt dient deze voorafgaand aan het gebruik te ondertekenen. In de huidige versie is hiervoor al een technische basis aanwezig in de applicatie (ComplianceCheckMiddleware en het ondertekeningsscherm), maar de inhoud van de documenten zelf vereist juridische expertise. (European Union, 2018) (Mailchimp, z.d.)

Lijst geraadpleegde werken

- Agile Business. (z.d.). *MOSCOW Prioritisation - DSDM Project Framework Handbook* | Agile Business Consortium. <https://www.agilebusiness.org/dsdm-project-framework/moscow-prioritisation.html>
- Alam, S. (2024, 16 september). *Difference between a framework and a library — never be confused again*. Medium. Geraadpleegd op 28 april 2026, van <https://medium.com/@sehban.alam/difference-between-a-framework-and-a-library-never-be-confused-again-9e3659f06dc1>
- Art. 17 GDPR – Right to erasure ('right to be forgotten') - General Data Protection Regulation (GDPR). (2017, 12 juni). General Data Protection Regulation (GDPR). <https://gdpr-info.eu/art-17-gdpr/>
- Auth. (2024, 30 november). *JSON Web Token Introduction - JWT.io*. JSON Web Tokens - jwt.io. <https://www.jwt.io/introduction#what-is-json-web-token>
- C10: Handle All Errors and Exceptions - OWASP Top 10 Proactive Controls. (z.d.). <https://top10proactive.owasp.org/archive/2018/c10-errors-exceptions/>
- Care and well-being - Research group Mobilab & Care | Thomas More. (z.d.). <https://thomasmore.be/en/exp-zw/mc>
- Clean Coder Blog. (2012, 13 augustus). <https://blog.cleancoder.com/uncle-bob/2012/08/13/the-clean-architecture.html>
- GeeksforGeeks. (2025, 23 juli). *React vs React Dom*. GeeksforGeeks. <https://www.geeksforgeeks.org/reactjs/react-vs-react-dom/>
- GeeksforGeeks. (2026, 31 maart). *Authentication vs authorization*. GeeksforGeeks. <https://www.geeksforgeeks.org/computer-networks/difference-between-authentication-and-authorization/>
- Home | XUnit.net. (z.d.). <https://xunit.net/?tabs=cs>
- How to send SMTP email using MailKit - SmarterASP.net. (z.d.). <https://www.smarterasp.net/support/kb/a2272/how-to-send-smtp-email-using-mailkit.aspx>
- HTML5 Security - OWASP Cheat Sheet Series. (z.d.). https://cheatsheetseries.owasp.org/cheatsheets/HTML5_Security_Cheat_Sheet.html
- Introducing asynchronous JavaScript - Learn web development | MDN. (2026, 11 februari). https://developer.mozilla.org/en-US/docs/Learn_web_development/Extensions/Async_JS/Introducing
- Joshi, M. (2025, 18 december). *Angular vs React vs Vue: Core Differences* | BrowserStack. BrowserStack. <https://www.browserstack.com/guide/angular-vs-react-vs-vue#:~:text=Angular%20excels%20in%20large%2Dscale,a%20seamless%20end%2Duser%20experience.>
- Kollegger, E. (2018, 14 mei). *What is Axios.js and why should I care?* Medium. Geraadpleegd op 28 april 2026, van <https://medium.com/@MinimalGhost/what-is-axios-js-and-why-should-i-care-7eb72b111dc0>
- Lawson, K. (2026, 26 februari). *What Is a Single Page Application?* Bloomreach. <https://www.bloomreach.com/en/blog/what-is-a-single-page-application>
- Maze, C., & Maze, C. (2024a, april 4). *CQRS and MediatR in ASP.NET Core*. Code Maze. <https://code-maze.com/cqrs-mediatr-in-aspnet-core/>
- Maze, C., & Maze, C. (2024b, april 4). *CQRS and MediatR in ASP.NET Core*. Code Maze. <https://code-maze.com/cqrs-mediatr-in-aspnet-core/>
- .NET - Build modern apps and powerful cloud services. (z.d.). Microsoft. <https://dotnet.microsoft.com/en-us/>

npm. (2026, 20 april). *React OAuth2* | Google. Npmjs. Geraadpleegd op 28 april 2026, van <https://www.npmjs.com/package/@react-oauth/google>

Okenyi, U. (2024, 26 januari). *CQRS with MediatR in ASP.NET Core: A Practical Guide to Decoupled Architecture*. Medium. Geraadpleegd op 28 april 2026, van <https://medium.com/@ulomaobilokenyi/cqrs-with-mediatr-in-asp-net-core-a-practical-guide-to-decoupled-architecture-7398f7eec846>

Oloruntoba, S. (2025a, juni 12). *SOLID Design Principles explained: Building better software architecture*. DigitalOcean. <https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design>

Oloruntoba, S. (2025b, juni 12). *SOLID Design Principles explained: Building better software architecture*. DigitalOcean. <https://www.digitalocean.com/community/conceptual-articles/s-o-l-i-d-the-first-five-principles-of-object-oriented-design>

Privacy policy for websites managed by the European Commission. (z.d.). European Commission. https://commission.europa.eu/privacy-policy-websites-managed-european-commission_en

Recharts. (z.d.). <https://recharts.github.io/>

Rm, A., & Rm, A. (2023, 10 januari). Why .NET is a Great Choice for Web Application Backend Development - CZ App Studio. *CZ App Studio - Smart Open Source Applications for Businesses*. Geraadpleegd op 28 april 2026, van <https://czappstudio.com/2023/01/10/why-net-is-a-great-choice-for-web-application-backend-development/>

Tailwind CSS - Rapidly build modern websites without ever leaving your HTML. (z.d.). Tailwind CSS. <https://tailwindcss.com/>

Tdykstra. (z.d.). *Overview of ASP.NET Core*. Microsoft Learn. <https://learn.microsoft.com/en-us/aspnet/core/overview?view=aspnetcore-8.0>

Team, O. A. S. P. (z.d.-a). *API2:2023 Broken Authentication - OWASP API Security Top 10*. <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>

Team, O. A. S. P. (z.d.-b). *API2:2023 Broken Authentication - OWASP API Security Top 10*. <https://owasp.org/API-Security/editions/2023/en/0xa2-broken-authentication/>

Tonka, O. (2023, 14 december). *What Is Separation Of Concerns?* Medium. Geraadpleegd op 28 april 2026, van <https://medium.com/@okay.tonka/what-is-separation-of-concern-b6715b2e0f75>

Van der Aalsvoort, J. (2024, 30 december). *The Power of Interfaces*. Medium. Geraadpleegd op 28 april 2026, van <https://medium.com/@jorisvdaalsvoort/the-power-of-interfaces-d1c48d6883c5>

Votre site Web a-t-il réellement besoin de conditions d'utilisation ? | Mailchimp. (z.d.). Mailchimp. <https://mailchimp.com/fr/resources/website-terms-of-use/>

Wikipedia contributors. (2026, 23 maart). *Object-oriented programming*. Wikipedia. https://en.wikipedia.org/wiki/Object-oriented_programming

WSTG - Latest | OWASP Foundation. (z.d.). https://owasp.org/www-project-web-security-testing-guide/latest/4-Web_Application_Security_Testing/04-Authentication_Testing/03-Testing_for_Weak_Lock_Out_Mechanism

Zorg en welzijn. (z.d.). Thomas More. <https://thomasmore.be/nl/onderzoek/expertisecentra/zorg-en-welzijn>

Figurenlijst

Figuur 1 CreatePatientModel.jsx	7
Figuur 2: Frontend Architectuur – schematische weergave	8
Figuur 3: Privacy Policy Flow	10
Figuur 4 Backend architectuur – schematische weergave	15
Figuur 5: ERD – relatie Patiënt en Sessie	18
Figuur 6: Use Case - Patiënt aanmaken	20
Figuur 7: Flow Diagram – Patiëntenpagina	24

Tabellenlijst

Tabel 1 Vergelijking Vue, React, Angular	6
Tabel 2 Rollen	9
Tabel 3 Codeerconventies	11
Tabel 4: Entiteit versus DTO	14
Tabel 5 MoSCow	21
Tabel 6: Toegang per rol - patiëntenpagina	22
Tabel 7 Patiënt Aanmaken	22
Tabel 8 Patiënt verwijderen	22
Tabel 9 Veelgemaakte fouten	22
Tabel 10 Planning	25

Bijlagen

Bijlage 1: 2-Technical_Documentation_Frontend

Bijlage 2: 3-Technical_Documentation_Backend

Bijlage 3: 11-Research

Bijlage 4: 5-Use_Cases

Bijlage 5: 9-Flowchart

Bijlage 6: UML_Model

Verklarende begrippen

API (Application Programming Interface): De interface waarlangs de frontend en backend met elkaar communiceren. Een gestandaardiseerde manier om verzoeken te sturen en antwoorden te ontvangen.

CQRS (Command Query Responsibility Segregation): Een ontwerppatroon dat lees- en schrijfoperaties strikt van elkaar scheidt.

Docker / Dockercontainer: Een technologie die een applicatie samen met al haar afhankelijkheden verpakt in een gestandaardiseerde eenheid, zodat ze op elke machine op dezelfde manier draait.

DOM (Document Object Model): De boomstructuur die een browser intern gebruikt om een webpagina voor te stellen. JavaScript kan deze structuur lezen en aanpassen.

ERD (Entity Relationship Diagram): Een visuele voorstelling van de structuur van een database: welke tabellen er zijn, welke gegevens ze bevatten en hoe ze met elkaar verbonden zijn.

GDPR (General Data Protection Regulation): De Europese privacywetgeving die regelt hoe persoonsgegevens verzameld, opgeslagen en verwerkt mogen worden.

Hashing: Een wiskundige bewerking die een invoer omzet naar een vaste, onleesbare reeks tekens. Wordt gebruikt voor de veilige opslag van wachtwoorden.

JWT (JSON Web Token): Een digitaal ondertekend token dat de identiteit en rol van een gebruiker bevat. Wordt gebruikt voor authenticatie en autorisatie.

MoSCoW: Een prioriteringstechniek waarbij vereisten worden ingedeeld in Must Have, Should Have, Could Have en Won't Have.

OOP (Object-Oriented Programming): Een programmeerstijl waarbij code wordt opgebouwd uit objecten die elk hun eigen data en functies bezitten.

OWASP (Open Worldwide Application Security Project): Een non-profitorganisatie die richtlijnen en best practices voor webbeveiliging publiceert.

SPA (Single Page Application): Een webapplicatie die bestaat uit één HTML-pagina. In plaats van bij elke actie een nieuwe pagina te laden, wordt enkel het relevante deel van de interface vervangen.

Use case: Een methodologie uit systeemanalyse die beschrijft wie een functie kan uitvoeren, onder welke voorwaarden en hoe het systeem reageert.

XSS (Cross-Site Scripting): Een aanvalstechniek waarbij een aanvaller kwaadaardige JavaScript-code in een webpagina injecteert om data te stelen of acties uit te voeren namens de gebruiker.

Verantwoording gebruik van AI

De inhoud, structuur en bronnen van dit document zijn volledig door mijzelf opgesteld. Artificiële intelligentie werd uitsluitend ingezet als schrijfhulp voor het verfijnen van zinsbouw en schrijfcontrole. Alle inhoudelijke keuzes, argumentatie, conclusies en inzichten zijn van mijzelf.